

TEMPERATURE FORECAST

3 HOURS AHEAD ON FPGA

A linear model from past values



A testable forecasting task

ASSIGNED TOPIC

Weather Pattern Forecasting on FPGA using Time Series Analysis



INPUT

Hourly temperature
at one location, up to time t



OUTPUT

One temperature forecast at $t + 3$ h
Unit: $^{\circ}\text{C}$

Why a linear model?

Clear arithmetic; C++ and RTL can be compared within 3 months.

Core: one variable, fixed coefficients. Add humidity/pressure only if time allows.
FPGA is for DSP/RTL practice; no speedup claim for hourly data.

C++ trains; FPGA forecasts

OFFLINE / PC



Clean → baselines → fit coefficients → fixed-point reference + test vectors

RUNTIME / COEFFICIENTS FIXED



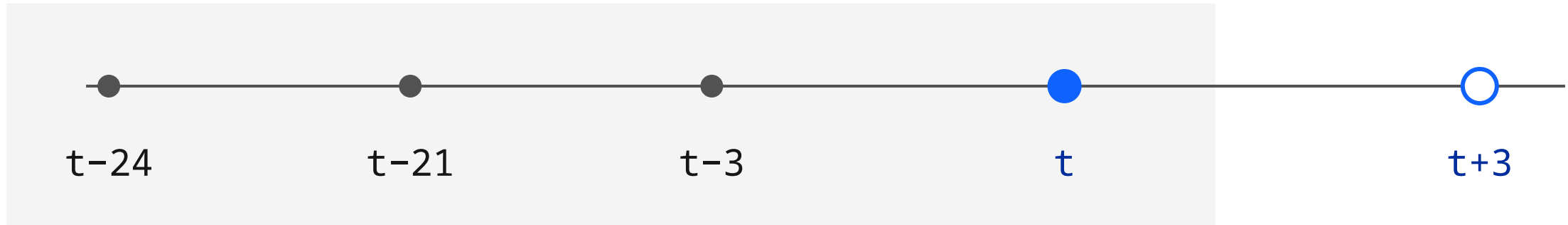
Store history · compute · output

DELIVERABLES: C++ · RTL · testbench · board demo · error/resource report

Four past values, one +3 h target

Planned source: **NASA POWER • hourly T2M**

Gridded data; location and date range TBD. Check missing/duplicate hours, time zone, and units.



$$t + 3 - 24 = t - 21$$

t-21 is the same hour on the day before the target.

Keep 25 consecutive samples. FPGA sees data through t; T(t+3) is for evaluation.
Timeline shows order, not time intervals to scale.

A linear model from past values

$$\hat{T}(t+3) = T(t-21) + a[T(t)-T(t-24)] \\ + b[T(t)-T(t-3)] + c$$

$T(t-21)$

Same-hour anchor
on the previous day

$a[T(t)-T(t-24)]$

Correction for day-to-day
difference

$b[T(t)-T(t-3)]$

Correction for recent
3-hour change

c is a constant offset. C++ fits a , b , c offline; FPGA uses them unchanged.
Proposed model only; no fitted coefficients or measured results yet.

How is one forecast computed?

ILLUSTRATIVE EXAMPLE — values below are not measured results.

$$T(t-24)=19^{\circ}\text{C} \quad T(t-21)=21^{\circ}\text{C} \quad T(t-3)=18^{\circ}\text{C} \quad T(t)=20^{\circ}\text{C}$$

Example coefficients: $a = 0.5$; $b = 0.2$; $c = 0^{\circ}\text{C}$

$$21 + 0.5 \times (20 - 19) + 0.2 \times (20 - 18)$$

$$= 21 + 0.5 + 0.4$$

21.9°C

Hardware operations: **subtract** → **multiply** → **add**

Train on the past, test on the future

$$x_1 = T(t) - T(t-24)$$

$$x_2 = T(t) - T(t-3)$$

$$y = T(t+3) - T(t-21)$$

$$y = a*x_1 + b*x_2 + c$$

C++ fits a , b , c by least squares.

01 TRAIN

Fit coefficients
on earlier data

02 VALIDATION

Select features
and settings

03 TEST

Final evaluation
on later data

Training labels cannot cross into validation/test; observed history remains usable.
Split chronologically; ratios TBD. C++ may use Eigen for least squares.

Two evaluations, two questions

IS THE FORECAST USEFUL?

Compare on the same test set:

Current value: $T(t)$

Same hour yesterday: $T(t-21)$

Proposed model

MAE (°C)

Mean absolute forecast error.

No results yet.

HARDWARE CORRECT?

Float ↔ fixed

Measure quantization error.

RTL ↔ fixed reference

Bit-exact with valid and latency.

LUT / FF / DSP / latency

From synthesis results.

Error limits will be set after experiments.

Fixed-point starts with value ranges

C++ FLOAT

Model's
reference

ANALYZE BEFORE CHOOSING BITS

Temperature · coefficient · intermediate ranges

Q format TBD until data and coefficients are known.

C++ FIXED AND RTL FOLLOW ONE SPEC

Signedness · fractional bits · product and sum widths

Shifts · rounding · overflow: saturation or wrap

C++ reference mirrors each RTL arithmetic step.

Test negative temperatures, numeric limits, and overflow.

Forecast error vs. truth and fixed-vs-float error are separate.

From sample history to arithmetic

Receive $T(t)$

25 SAMPLES

$T(t-24) \dots T(t)$

Required taps:

$t-24$	$t-21$
$t-3$	t

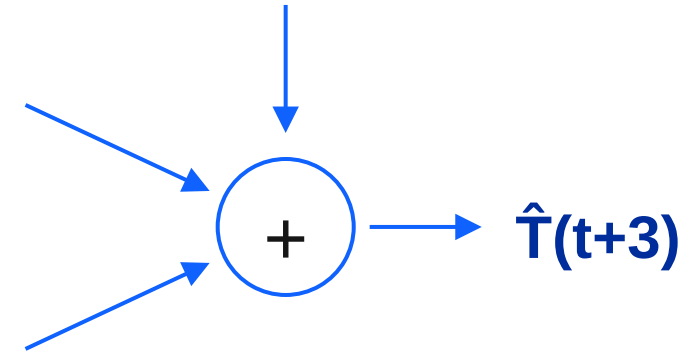


$a \times \Delta_{\text{day}}$



$b \times \Delta_{3h}$

$T(t-21), c$



Functional diagram; multiplier layout TBD.

Clock + valid accept samples. Forecast only after 25 valid samples.

Coefficients stay fixed; choose shared or parallel multipliers after board selection.

One sample delay = 1 data hour. Hardware latency is in clock cycles, not 3 hours.

Three verification layers

01 / C++

FLOAT ↔ FIXED

Measure error
from quantization

Test:
negative temps, limits,
rapid changes.

02

TB ↔ RTL

Compare each output
with valid and latency

Target: bit-exact.
Test reset, overflow,
warm-up, valid gaps.

03

BOARD ↔ REF.

Check interfaces
and integrated output

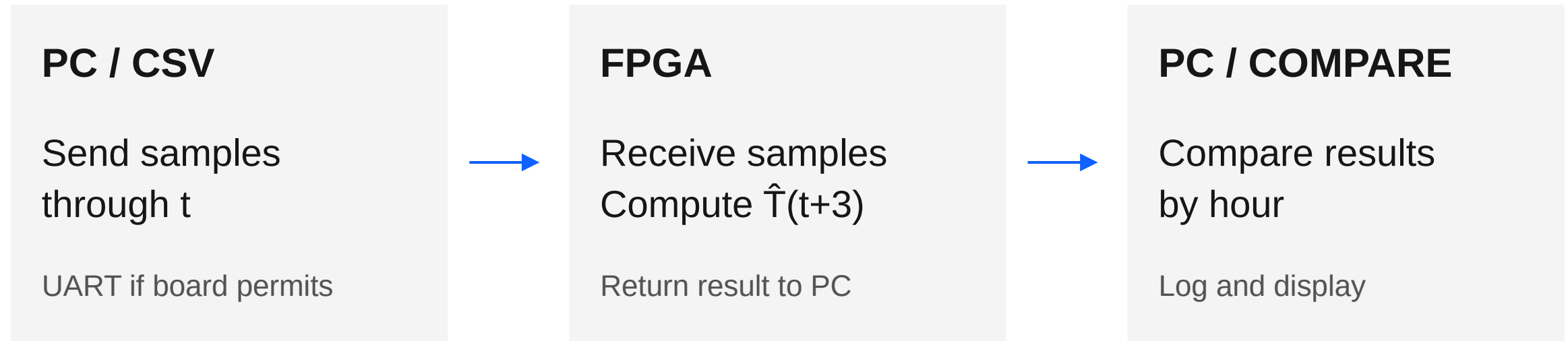
Use the same vectors;
check sample order,
input and output logs.

Same data and arithmetic rules; each layer answers a different question.

Demo by replaying historical data

Replay quickly while preserving time order.

Proposed demo; no board run or live interface yet.



PC holds $T(t+3)$ for evaluation; future labels never enter FPGA.
Board, interface, and minimum demo scope need instructor input.

12 weeks, four milestones

WEEKS 1–3



WEEKS 4–6



WEEKS 7–9



WEEKS 10–12



Data

Learn basics
Clean CSV
Two baselines

C++ model

Fit + validate
Select features
Start fixed-point

RTL + testbench

Arithmetic core
Bit comparison
Prepare integration

Board + report

Demo + test
Hardware resources
Limits and conclusions

Each phase has a working deliverable; session 2 follows the instructor's schedule.

Three students, coordinated outputs

STUDENT 1

Data + C++

CSV · baselines
Coefficients · MAE · reference

RISK: MISSING HOURS

Validate and repair hours
before building samples.

STUDENT 2

Fixed-point + RTL

Buffer · arithmetic core
Synthesis · board integration

RISK: BOARD TBD

Build a standalone core
and simulate it first.

STUDENT 3

Verification + demo

Vectors · bit checking
I/O · result logs

RISK: HIGH ERROR

Revisit validation together;
report test results honestly.

Agree on data, coefficients, signedness, rounding/overflow, valid, and latency.

One forecast, two claims to verify

Linear model → temperature +3 h

01 Forecast quality

Compare with baselines on unseen data.

02 Hardware correctness

RTL matches the fixed-point reference.

Instructor input: +3 h scope · board/tools · minimum demo.

Add humidity/pressure only after the core is complete.